

MLOps: Heart Disease Prediction

End-to-End Production Deployment Architecture

BITS WILP •
S1-25_AIMLCZG523

Assignment I: Scalable ML Model Development &

Deployment

S.No.	Team Member Name	BITS ID
1	Biprajit Debnath	2024AA05762
2	Bavya M	2024AA05761
3	HARIVIGNESH K S	2024AA05765
4	MATURI HAREESH	2024aa05211
5	VAIBHAV V SIMIKERI	2024aa05722

Project Overview & Objectives

- **Dataset:** Heart Disease UCI (14+ features, binary classification)
- **Problem:** Predict heart disease risk from patient health data
- **Goal:** Deploy production-ready, monitored ML API
- **Stack:** Python, Flask, Docker, Jenkins, Airflow, Prometheus, Grafana

Development Pipeline: EDA & Model Building

EDA Phase

Data cleaning, handling missing values, feature encoding, correlation analysis, class balance visualization

Model Development

Logistic Regression & Random Forest, hyperparameter tuning, cross-validation, metrics: Accuracy, Precision, Recall, ROC-AUC

Experiment Tracking

MLflow integration for logging parameters, metrics, artifacts, and model runs

Model Packaging

Package model as Joblib with integrated preprocessing pipeline for full reproducibility

CI/CD Pipeline: Automation & Testing

GitHub → Jenkins Workflow

Code push triggers automated pipeline: Code Checkout → Linting → Unit Tests (Pytest) → Model Training → Build Docker Image → Push to DockerHub

Testing Strategy

Unit tests for data preprocessing and model inference. Pipeline fails on code/test errors with clear logs. Artifact logging for each workflow run.

Orchestration: Airflow for Model Changes

Continuous Training (CT) Pipeline

- ✓ Triggers on schedule or data drift detection
- ✓ Executes EDA → Feature Engineering → Model Training → Validation
- ✓ Packages model as Joblib for production export
- ✓ Logs experiments to MLflow for tracking

Containerization & Deployment

Docker Build

- Flask API with /predict endpoint
- Loads model.joblib & scaler.joblib
- Accepts JSON input, returns prediction & confidence
- Dockerfile optimized for production

Registry & Runtime

- Image pushed to DockerHub
- Docker Desktop runs container (CD)
- Service exposed on port 5007
- Verified with sample inputs

Monitoring & Observability Stack

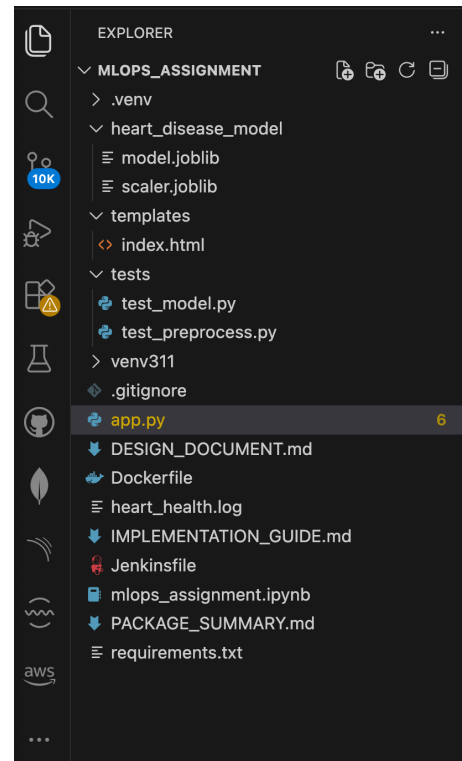
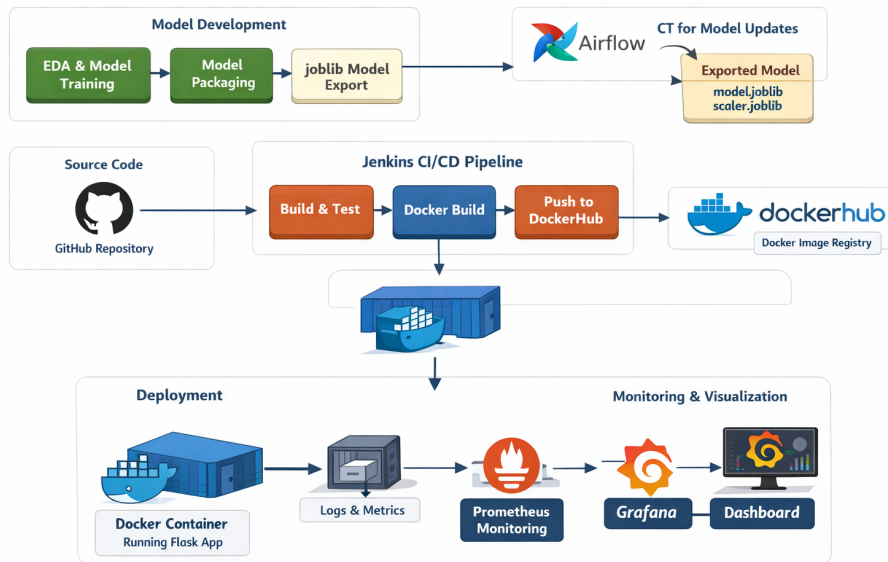
Logging & Metrics

Container logs capture API requests, predictions, model loading, and errors. Prometheus client integrated to export metrics from Flask app.

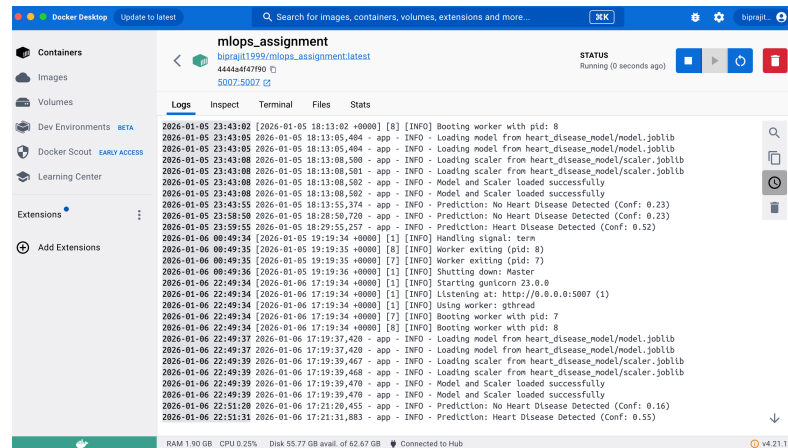
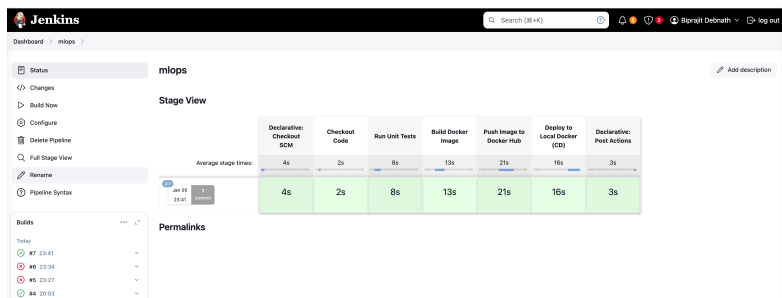
Prometheus & Grafana

Prometheus scrapes metrics endpoint. Grafana dashboards visualize request rates, latency, prediction distribution, and system health metrics.

Complete MLOps Architecture



Deployments Screenshots



Deliverables & Repository Structure

GitHub Repository Contains:

- Code, Dockerfile, requirements.txt, dataset download script (
- Jupyter notebooks (EDA, training, inference), test/ folder with unit tests
- GitHub Actions/Jenkinsfile, Deployment manifests/Helm charts
- Screenshot folder for reporting, Final 10-page written report (doc/docx)

Additional Deliverables:

- End-to-end pipeline demonstration video
- Deployed API URL or local testing instructions
- CI/CD pipeline screenshots (Jenkins stages, Docker build logs)
- Monitoring dashboard screenshots (Prometheus, Grafana)
- Github Link :- http://github.com/biprajit1999/mlops_assignment_grp123
- DockerHub Link: - https://hub.docker.com/repository/docker/biprajit1999/mlops_assignment/general
- Youtube Video Link:- <https://www.youtube.com/watch?v=maaUxvUeJ1w>

Documentation & Reporting (Point 9)

Professional Report (10 Pages) Includes:

- Setup/Install Instructions with requirements.txt & environment configuration
- EDA Summary: data characteristics, distributions, class balance, feature importance
- Modeling Choices: model selection rationale, hyperparameter tuning, evaluation metrics
- Experiment Tracking: MLflow runs, parameters logged, artifacts, performance comparison
- Architecture Diagram: end-to-end pipeline visualization (ML → CI/CD → Deploy → Monitor)
- CI/CD Workflow: GitHub/Jenkins pipeline screenshots, Docker build & push logs
- Deployment Screenshots: Docker Desktop running container, Flask API logs, test predictions
- Monitoring Dashboard: Prometheus targets, Grafana dashboards, metrics visualization
- Link to Code Repository: GitHub repository URL with all source code
- Production-Readiness Checklist: scripts execute from clean setup, Docker isolation verified, pipeline error handling confirmed